



中华人民共和国国家标准

GB/T 34944—2017

Java 语言源代码漏洞测试规范

Source code vulnerability testing specification for Java

2017-11-01 发布

2018-05-01 实施

中华人民共和国国家质量监督检验检疫总局
中国国家标准化管理委员会 发布

目 次

前言 I

引言 II

1 范围 1

2 规范性引用文件 1

3 术语和定义 1

4 缩略语 4

5 源代码漏洞测试总则 4

 5.1 源代码漏洞测试目的 4

 5.2 源代码漏洞测试过程 4

 5.3 源代码漏洞测试管理 5

 5.4 源代码漏洞测试工具 7

 5.5 源代码漏洞测试文档 7

6 源代码漏洞测试内容 7

 6.1 源代码漏洞分类 7

 6.2 源代码漏洞说明 8

附录 A（资料性附录） Java 语言源代码漏洞测试案例 50

附录 B（资料性附录） Java 语言源代码漏洞类别与名称 56

参考文献 58

前 言

本标准按照 GB/T 1.1—2009 给出的规则起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本标准由全国信息技术标准化技术委员会(SAC/TC 28)提出并归口。

本标准起草单位:珠海南方软件网络评测中心、杭州安恒信息技术有限公司、厦门理工学院、上海端玛计算机科技有限公司、中国电子技术标准化研究院、东信和平科技股份有限公司、远光软件股份有限公司、南京大学、国家应用软件产品质量监督检验中心、珠海中慧微电子有限公司、广东省科技基础条件平台中心、珠海市软件行业协会、南昌金庐软件园软件评测培训有限公司、吉林省电子信息产品监督检验研究院。

本标准主要起草人:侯建华、黄兆森、王忠福、范渊、杨尚沅、邓人逊、梁建新、张旻旻、李军、李璐、王威、黄华婕、刘早、辛士界、陈振宇、肖泉、崔建峰、申煜湘。

引 言

Java 语言是一种面向对象的、运行于 Java 虚拟机之上的高级程序设计语言。它广泛应用于各种大型信息系统和智能终端应用软件的开发。众所周知,由于各种人为因素影响,每个软件的源代码都难免会存在漏洞,而软件信息泄露、数据或代码被恶意篡改等安全事件的发生一般都与源代码漏洞有关。为尽量减少 Java 语言源代码中存在的漏洞,有必要制定针对 Java 语言程序的源代码漏洞测试规范。

源代码漏洞测试可在开发过程的软件编码活动之后实施,也可在运行和维护过程中实施。

本标准的漏洞分类与漏洞说明主要参考了 MITRE 公司发布的 CWE(Common Weakness Enumeration),同时结合了当前行业主流的自动化静态分析工具在测试实践中发现的典型漏洞来确定并进行说明。

注:本标准漏洞参考了 CWE2.9 版本,示例代码适用于本标准选择的漏洞说明。

本标准仅针对自动化静态分析工具支持的关键漏洞进行说明,应用本标准开展源代码漏洞测试时应根据实际需要对漏洞进行裁剪和补充。

Java 语言源代码漏洞测试规范

1 范围

本标准规定了 Java 语言源代码漏洞测试的测试总则和测试内容。

本标准适用于开发方或第三方机构的测试人员利用自动化静态分析工具开展的 Java 语言源代码漏洞测试活动,Java 语言的程序设计和编码人员以及源代码漏洞测试工具的设计人员也可参考使用。

2 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件,仅注日期的版本适用于本文件。凡是不注日期的引用文件,其最新版本(包括所有的修改单)适用于本文件。

GB/T 11457 信息技术 软件工程术语

GB/T 15532—2008 计算机软件测试规范

GB/T 20158—2006 信息技术 软件生存周期过程 配置管理(ISO/IEC TR 15846:1998,IDT)

3 术语和定义

GB/T 11457 界定的以及下列术语和定义适用于本文件。

3.1

访问控制 access control

一种保证数据处理系统的资源只能由被授权主体按授权方式进行访问的手段。

[GB/T 25069—2010,定义 2.2.1.42]

3.2

攻击 attack

在信息系统中,对系统或信息进行破坏、泄露、更改或使其丧失功能的尝试(包括窃取数据)。

[GB/T 25069—2010,定义 2.2.1.58]

3.3

密码分组链接 cipher block chaining

对信息加密时,每一密文块在加密时都依赖于前一密文块的方式。

3.4

密文 ciphertext

利用加密技术,经变换,信息内容被隐藏起来的数据。

[GB/T 25069—2010,定义 2.2.2.105]

3.5

解密 decryption

将密文转换为明文的处理,即加密对应的逆过程。

[GB/T 25069—2010,定义 2.2.2.69]

3.6

字典式攻击 dictionary attack

用遍历给定口令或密钥列表的方式对密码系统的攻击。如,使用存储的特定口令值或密钥值列表,

或使用来自自然语言字典中的单词列表。

[GB/T 25069—2010, 定义 2.2.2.239]

3.7

域名服务器(DNS)欺骗 domain name server(DNS) spoofing

攻击者冒充域名服务器的一种欺骗行为。

3.8

加密 encryption

对数据进行密码变换以产生密文的过程。一般包含一个变换集合,该变换使用一套算法和一套输入参量。输入参量通常被称为密钥。

[GB/T 25069—2010, 定义 2.2.2.60]

3.9

硬编码 hard-coding

在软件实现上,把输入或输出的相关参数直接编码在源代码中,而不是从外部来源获得的数据生成的。

3.10

散列值 hash-value

散列/杂凑函数的输出的比特串。

[GB/T 25069—2010, 定义 2.2.2.169]

3.11

散列/杂凑函数 hash function

将比特串映射为固定长度的比特串的函数,该函数满足下列两特性:

——对于给定输出,找出映射为该输出的输入,在计算上是不可行的;

——对于给定输入,找出映射为同一输出的第二个输入,在计算上是不可行的。

注:计算上的可行性取决于特定安全要求和环境。

[GB/T 25069—2010, 定义 2.2.2.166]

3.12

注入 injection

由于字符过滤不严谨导致的漏洞。

3.13

单向加密 one-way encryption

一种加密,它只产生密文,而不能将密文再生为原始数据。

[GB/T 25069—2010, 定义 2.2.2.9]

3.14

最优非对称加密填充 optimal asymmetric encryption padding

RSA 算法的一种加密填充方案,可将原有的确定性的加密方案转换成一种可能性方案,并能防止密文的部分解密或其他信息泄露。

3.15

填充 padding

对数据串附加额外比特。

[GB/T 25069—2010, 定义 2.2.2.187]

3.16

口令 password

用于身份鉴别的秘密的字、短语、数字或字符序列,通常是被默记的弱秘密。

[GB/T 25069—2010, 定义 2.2.2.76]

3.17

明文 plaintext

未加密的信息。

[GB/T 25069—2010, 定义 2.2.2.135]

3.18

彩虹表 rainbow table

一个用于加密散列函数逆运算的预先计算好的表, 常用于破解加密过的密码散列。

3.19

重定向 redirect

通过服务器的特殊设置, 将访问当前域名的用户引导到指定的另一个网络地址, 即将一个域名指向另一个已经存在的站点, 或者将一个页面引导至另一个页面。

3.20

反向域名解析 reverse DNS resolution

通过 IP 地址查询服务器域名。

3.21

盐值 salt

作为单向函数或加密函数的二次输入而加入的随机变量, 可用于导出口令验证数据。

3.22

脚本 script

使用一种特定的描述性语言, 依据一定的格式编写的纯文本程序。

3.23

种子 seed

一种用作某一确定性随机比特生成器输入的比特串。DRBG 的部分状态由种子确定。

[GB/T 25069—2010, 定义 2.2.2.232]

3.24

敏感信息 sensitive information

由权威机构确定的必须受保护的信息, 该信息的泄露、修改、破坏或丢失会对人或事产生可预知的损害。

[GB/T 25069—2010, 定义 2.2.4.7]

3.25

序列化 serialization

将对象的状态信息转换成可以存储或传输的字节序列的过程。

3.26

会话 session

用于跟踪客户端与服务器间的交互, 保存用户身份和信息的对象。

3.27

源代码漏洞 source code vulnerability

存在于软件源代码中的漏洞。

3.28

漏洞 vulnerability

计算机信息系统在需求、设计、实现、配置、运行等过程中, 有意或无意产生的缺陷。这些缺陷以不同形式存在于计算机信息系统的各个层次和环节之中, 如果利用不当, 就会对计算机信息系统的安全造

成损害,从而影响计算机信息系统的正常运行。

注: 改写 GB/T 28458—2012,定义 3.2。

4 缩略语

下列缩略语适用于本文件。

- DNS:域名服务器(Domain Name Server)
- DRBG:确定性随机比特生成器(Deterministic Random Bit Generator)
- HTTP:超文本传输协议(Hypertext Transfer Protocol)
- HTTPS:安全套接字层超文本传输协议(Hypertext Transfer Protocol Over Secure Socket Layer)
- IP:网际协议(Internet Protocol)
- PRNG:伪随机数生成器(Pseudorandom Number Generator)
- SQL:结构化查询语言(Structured Query Language)
- TCP:传输控制协议(Transmission Control Protocol)
- URL:统一资源定位符(Uniform Resoure Locator)
- XML:可扩展置标语言(Extensible Markup Language)

5 源代码漏洞测试总则

5.1 源代码漏洞测试目的

源代码漏洞测试的目的是:

- a) 发现、定位及解决软件源代码中的漏洞;
- b) 为软件产品的安全性测量和评价提供依据。

5.2 源代码漏洞测试过程

5.2.1 概述

源代码作为软件产品的重要组成部分,其测试过程基本等同软件产品的测试过程。本标准遵循 GB/T 15532—2008 的要求将源代码测试过程分为测试策划、测试设计、测试执行和测试总结四个阶段。

5.2.2 测试策划

测试策划主要对整个源代码漏洞测试的过程进行策划。测试策划应确定测试的目标、范围、依据、环境和工具,应分析与评估测试风险,并制定应对措施。测试策划应重点明确源代码漏洞测试应划分的阶段以及各阶段的人员角色、任务、时间和工作成果,形成源代码漏洞测试进度计划表。源代码漏洞测试进度计划见表 1。

表 1 源代码漏洞测试进度计划

测试阶段	人员角色	任务	时间	工作成果
测试策划				《测试计划》
测试设计				《测试说明》
测试执行				《测试日志》
测试总结				《测试总结报告》

5.2.3 测试设计

测试设计应根据测试目标,结合被测源代码的业务和技术特点,明确测试环境和工具,确定测试需求、测试方法、测试内容、测试准入条件和测试准出条件。测试方法应采用自动化静态分析工具扫描和人工分析相结合的方法。Java 语言源代码漏洞测试的测试内容宜包括但不限于以下源代码漏洞分类:

- a) 行为问题;
- b) 路径错误;
- c) 数据处理;
- d) 处理程序错误;
- e) 不充分的封装;
- f) 安全功能;
- g) 时间和状态;
- h) Web 问题;
- i) 用户界面错误。

以上 9 类源代码漏洞分别对应 6.1 源代码漏洞相关分类。

若被测源代码采用了 Java 语言的第三方框架,测试人员应根据被测源代码的实际情况在测试内容中增加第三方框架相关的漏洞。

应设计测试用例。源代码漏洞测试的测试用例应包括但不限于以下要素:

- a) 名称和编号;
- b) 自动化静态分析工具的操作步骤和参数配置;
- c) 自动化静态分析工具的期望操作结果。

5.2.4 测试执行

测试执行包括自动化静态分析工具扫描和人工分析。

应根据测试用例明确的操作步骤,使用自动化静态分析工具执行测试,记录测试执行过程及测试结果。

应对自动化静态分析工具的测试结果进行人工分析,人工分析宜包括但不限于以下任务:

- a) 宜按漏洞类别或漏洞风险级别从高到低的顺序分析扫描得到的所有源代码漏洞;
- b) 结合源代码的上下文和业务需求,验证疑似漏洞,筛除误报的源代码漏洞;
- c) 与开发人员沟通确认源代码漏洞分析结果。

5.2.5 测试总结

测试总结应对整个源代码漏洞测试过程进行总结,测试总结应包括但不限于以下任务:

- a) 核查测试环境、工具、内容、方法和结果是否正确;
- b) 确认测试目标和测试需求是否得到满足;
- c) 总结测试内容、方法和结果,出具测试报告。

5.3 源代码漏洞测试管理

5.3.1 过程管理

应对源代码漏洞测试的过程进行管理,一般包括:

- a) 提出源代码漏洞测试各个阶段的任务要求和质量要求。
 - b) 安排对源代码漏洞测试的过程进行质量监督和阶段评审,包括监督和评审所需的环境、设备、资金和人员,质量监督的记录应形成文档。
 - c) 对源代码漏洞测试的风险进行管理,提供风险规避所需的相关资源。
 - d) 提供完成源代码漏洞测试各项任务所需的资源保障。包括测试的环境、工具、资金和人员。
- 表 2 给出了源代码漏洞测试的人员配备的参考。

表 2 源代码漏洞测试人员配备情况表

工作角色	具体职责
测试项目负责人	管理监督测试项目,提供技术指导,获取适当的资源,制定基线,技术协调,负责项目的安全保密、过程管理和质量管理,负责测试策划和测试总结
测试设计员	开展测试需求分析,确定测试内容、测试方法、测试(软、硬件)环境、测试工具,设计测试用例,建立测试环境
测试员	执行测试,记录测试过程和结果
测试分析员	对自动化静态分析工具的测试结果进行人工分析
测试系统管理员	对测试环境和资产进行管理和维护
配置管理员	设置、管理和维护测试配置管理数据库
测试评审员	对测试的各个阶段进行评审
注 1: 当软件的供方实施测试时,配置管理员由软件开发项目的配置管理员承担;当独立的测试组织实施测试时,需配备测试活动的配置管理员。 注 2: 一个人可承担多个角色的工作,一个角色可由多个人承担。	

5.3.2 过程评审

5.3.2.1 概述

源代码漏洞测试包括测试策划、测试设计、测试执行和测试总结四个阶段。每个测试阶段结束时应开展阶段评审。评审的级别和参加人员要求宜根据被测源代码的重要程度而确定。

5.3.2.2 测试策划评审

完成测试策划后,应对测试计划进行评审,形成测试策划评审表。评审的具体内容应包括:

- a) 评审测试环境、工具等测试实施条件要素考虑是否全面合理;
- b) 评审测试人员分工和进度计划等测试组织要素是否具有可实施性;
- c) 评审风险分析是否全面合理以及是否具有可行的应对风险的措施。

5.3.2.3 测试设计评审

完成测试设计后,应对测试说明和测试就绪情况进行评审,形成测试设计评审表。评审的具体内容应包括:

- a) 评审测试需求分析是否合理;
- b) 评审测试内容和方法是否符合测试需求;

- c) 评审测试用例的操作步骤和参数配置是否详细、正确、可实施；
- d) 评审测试用例的期望结果描述是否准确；
- e) 评审测试人员、环境和工具是否齐备并符合要求。

5.3.2.4 测试执行评审

完成测试执行后,应对测试日志进行评审,形成测试执行评审表。评审的具体内容应包括:

- a) 评审测试用例执行是否完整；
- b) 评审操作结果是否真实有效；
- c) 评审操作结果描述是否清晰、准确；
- d) 对于与预期结果不一致的操作结果,评审是否记录了详细的问题现象；
- e) 评审人工分析的结果是否正确。

5.3.2.5 测试总结评审

完成测试总结后,应对测试总结报告进行评审,形成测试总结评审表。评审的具体内容应包括:

- a) 评审测试需求和测试目标是否全面准确完成；
- b) 评审测试结论与测试结果追溯的合理性；
- c) 评审是否具备测试结束条件；
- d) 评审测试风险规避是否合理。

5.3.3 配置管理

应按照软件配置管理的要求,将测试过程中产生的各种软件工作产品纳入配置管理。配置管理要求见 GB/T 20158—2006。

5.4 源代码漏洞测试工具

选择源代码漏洞测试工具应首先明确 GB/T 15532—2008 中 4.8.2 的要求,重点应考虑工具的漏报率和误报率,可通过调查或比较的方式评估工具的漏报率和误报率。

选择的源代码漏洞测试工具应覆盖但不限于本标准的源代码漏洞测试内容,测试前应对工具的漏洞规则库和测试引擎进行必要的升级和维护。

注: 选择源代码漏洞测试工具建议结合项目的具体需求。如可行,首先考虑选用商业的自动化静态分析工具。若条件不具备,也可选用开源的自动化静态分析工具。

5.5 源代码漏洞测试文档

源代码漏洞测试文档一般包括测试计划、测试说明、测试日志和测试总结报告。源代码漏洞测试文档宜单独出具,也可与其他测试类型出具的测试文档合并。测试文档的基本内容参见附录 A 的案例。

6 源代码漏洞测试内容

6.1 源代码漏洞分类

本标准根据软件开发中常用的概念来组织 Java 语言源代码漏洞的类别,具体分为如下 9 个类别:

- a) 行为问题——由于应用程序的意外行为而引发的漏洞。
- b) 路径错误——不恰当的处理访问路径而引发的漏洞。

- c) 数据处理——处理数据的功能中发现的漏洞。
- d) 处理程序错误——由于处理程序的管理不当而引发的漏洞。
- e) 不充分的封装——未充分封装关键数据或功能而引发的漏洞。
- f) 安全功能——软件安全功能如身份鉴别、访问控制、机密性、密码学和特权管理等相关的漏洞。
- g) 时间和状态——在多系统、进程或线程并发计算的环境下由于时间和状态管理不恰当而引发的漏洞。
- h) Web 问题——Web 技术相关的漏洞。
- i) 用户界面错误——与用户界面相关的漏洞。

Java 语言各源代码漏洞的名称及其所属的类别参见附录 B。

6.2 源代码漏洞说明

6.2.1 行为问题

6.2.1.1 不可控的内存分配

漏洞描述:内存分配的大小受外部控制的输入数据影响,且程序没有指定内存分配大小的上限。

漏洞风险:攻击者可以使程序分配大量的内存,程序可能会因为内存资源不足而崩溃。

修复或规避建议:在程序中指定内存分配大小的上限,在分配内存前对要分配的内存大小进行验证,确保要分配的内存大小不超过上限。

示例 1: 不规范用法

```
public class Example{
    public int exampleFun(int length){ //length 为用户的输入数据
        String[] buffer;
        if(length < 0){ //没有验证 length 是否超出内存分配的上限
            return 0;
        }
        buffer = new String[length];
        ...
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    static int MAX_LENGTH = 1000; //指定内存分配的上限或动态判断剩余内存大小自动分配
    public int exampleFun(int length){ //length 为用户的输入数据
        if(length > MAX_LENGTH || length < 0){ //对 length 进行验证
            return 0;
        }
        String[] buffer = new String[length];
        ...
    }
}
```

6.2.2 路径错误

6.2.2.1 不可信的搜索路径

漏洞描述:程序使用关键资源时没有指定资源的路径,而是依赖操作系统去搜索资源。

漏洞风险:攻击者可以在搜索优先级更高的文件夹中放入相同名称的资源,程序会使用攻击者控制的资源。

修复或规避建议:使用关键资源时指定资源所在的路径。

示例 1: 不规范用法

```
public class Example{
    private String command; //本例中 command = "dir.exe E:\\data"
    public void exampleFun(void){
        //攻击者可在搜索优先级更高的文件夹中放入和 dir.exe 同名的恶意程序导致 command 的内容无法正
        确执行
        Runtime.getRuntime().exec(command);
        ...
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    private String command; //本例中 command = "dir.exe E:\\data"
    public void exampleFun(void){
        //PATH 是存放操作系统中 dir.exe 命令所在完整路径的常量,本例中 PATH="C:\\WINDOWS\\
        system32\\"
        String cmd = PATH + command; //使用完整路径确保 command 的内容能正确执行
        Runtime.getRuntime().exec(cmd);
        ...
    }
}
```

6.2.3 数据处理

6.2.3.1 相对路径遍历

漏洞描述:路径名受外部控制的输入数据影响,且程序没有使能够解析到目录外位置的字符序列(如“..”)失效。

漏洞风险:攻击者可以通过输入能够解析到目录外位置的字符序列来访问限制目录之外的文件或目录。

修复或规避建议:在构建路径名前对输入数据进行验证,确保外部输入仅包含允许的构成路径名的字符。

示例 1：不规范用法

```
import java.io.*;
public class Example{
    private String dataPath; //本例中允许访问的目录
    public void exampleFun(String filename){ //filename 为用户的输入数据,不超过 10 个字符
        //filename 可能包含“..”字符序列导致访问 dataPath 目录之外的文件
        String path = dataPath + filename;
        try{
            File file = new File(path);
            if(file.exists()){
                file.open();
                ...
            }
            ...
        }
        catch(...){
            ...
        }
    }
}
```

示例 2：规范用法之一

```
import java.io.*;
public class Example{
    private String dataPath; //本例中允许访问的目录
    public void exampleFun(String filename){ //filename 为用户的输入数据,不超过 10 个字符
        String regex = "[A-Za-z0-9]+\\. [a-z]+ $ "; //对文件名进行合法性校验的正则表达式
        //参数 filename 传入的文件名称可能存在漏洞,现通过正则表达式 regex 校验文件名 filename 的合法性
        //校验通过则访问文件
        if(filename.matches(regex)){
            //如果文件名合法,则将路径和文件名组合成完整的文件访问路径
            String path = dataPath + filename;
            try{
                //访问文件,这里的 path 已经是合法的、安全的文件路径,不存在漏洞
                File file = new File(path);
                if(file.exists()){
                    file.open();
                    ...
                }
                ...
            }
            catch(...){
                ...
            }
        }
    }
}
```

6.2.3.2 绝对路径遍历

漏洞描述:路径名由外部控制的输入数据决定,且程序没有限制路径名允许访问的目录。

漏洞风险:攻击者可以通过输入路径名来访问任意的文件或目录。

修复或规避建议:在程序中指定允许访问的文件或目录,在访问文件或目录前对路径名进行验证,确保仅允许访问指定的文件或目录。

示例 1: 不规范用法

```
import java.io.*;
public class Example{
    public void exampleFun(String absolutePath){ //absolutePath 为用户的输入数据
        try{
            File file = new File(absolutePath); //没有限制 absolutePath 访问的目录
            if(file.exists()){
                file.open();
                ...
            }
            ...
        }
        catch(...){
            ...
        }
    }
}
```

示例 2: 规范用法之一

```
import java.io.*;
public class Example{
    public void exampleFun(String absolutePath){ //absolutePath 为用户的输入数据
        String regex = "C:\\data\\[A-Za-z0-9]+\\.\\[a-z]+ $ "; //用正则表达式设置限制访问的目录
        //验证 absolutePath 是否仅能解析到受限制的目录内
        if(absolutePath != null && absolutePath.matches(regex)){
            try{
                File file = new File(absolutePath);
                if(file.exists()){
                    file.open();
                    ...
                }
                ...
            }
            catch(...){
                ...
            }
        }
        ...
    }
}
```

6.2.3.3 命令注入

漏洞描述:使用未经验证的输入数据构建命令。

漏洞风险:攻击者可执行任意命令。

修复或规避建议:在构建命令前对输入数据进行验证,确保输入数据仅能用于构成允许的命令。

示例 1: 不规范用法

```
public class Example{
    public void exampleFun(String parameter){ //command 为用户的输入数据
        String INIT_CMD = PATH + "cmd.exe /c \"dir.exe "; //PATH 是操作系统中 cmd.exe 所在的完整
        路径
        if(parameter != null){ //未对 parameter 进行验证,攻击者可输入"&&"等字符串执行恶意命令
            String cmd = INIT_CMD + parameter + "\"";
            Runtime.getRuntime().exec(cmd);
            ...
        }
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    public void exampleFun(String parameter){ // parameter 为用户的输入数据
        String INIT_CMD = PATH + "cmd.exe /c \"dir.exe "; //PATH 是操作系统中 cmd.exe 所在的完整
        路径
        String regex = "[\\w\\ ]+$ "; //设置白名单
        if(parameter != null && parameter.matches(regex)){ //验证 parameter
            String cmd = INIT_CMD + parameter + "\"";
            Runtime.getRuntime().exec(cmd);
            ...
        }
    }
}
```

6.2.3.4 SQL 注入

漏洞描述:使用未经验证的输入数据采用拼接字符串的方式形成 SQL 语句。

漏洞风险:攻击者可输入任何 SQL 语句,实现越权查询数据库的敏感数据、非法修改数据库数据和提升攻击者的数据库权限等目的。

修复或规避建议:在拼接 SQL 语句前对输入数据进行验证,确保输入数据不包含 SQL 语句的关键字符;或者采用 PreparedStatement 创建 SQL 语句,并将输入数据作为 SQL 语句的参数。

示例 1: 不规范用法

```

public class Example{
    public ResultSet getUserData(ServletRequest req, Connection con) throws SQLException {
        String owner = req.getParameter("owner");
        //采用拼接字符串的方式形成 SQL 语句,没有对用户输入数据 owner 进行验证
        String query = "SELECT * FROM user_data WHERE userid = " + owner + ";
        Statement statement = con.createStatement();
        ResultSet results = statement.executeQuery(query);
        return results;
    }
}

```

示例 2: 规范用法之一

```

public class Example{
    public ResultSet getUserData(ServletRequest req, Connection con) throws SQLException {
        String owner = req.getParameter("owner");
        //采用 PreparedStatement 的方式形成 SQL 语句
        PreparedStatement preStatement = con.prepareStatement("SELECT * FROM user_data WHERE
userid = ?");
        preStatement.setString(1, owner);
        ResultSet results = preStatement.executeQuery();
        return results;
    }
}

```

示例 3: 规范用法之一

```

public class Example{
    public ResultSet getUserData(ServletRequest req, Connection con) throws SQLException {
        String owner = req.getParameter("owner");
        //本例中假设用户名是一个 3 到 8 位的数字、字母或下划线组成的字符串
        String regex = "^(\w){3,8}$ "; //正则表达式
        if(owner != null && owner.matches(regex)){ //验证 owner 是否合法
            String query = "SELECT * FROM user_data WHERE userid = " + owner + ";
            Statement statement = con.createStatement();
            ResultSet results = preStatement.executeQuery();
            return results;
        }
        else{
            ...
        }
    }
}

```

6.2.3.5 代码注入

漏洞描述:使用未经验证的输入数据动态构建代码语句。

漏洞风险:攻击者可构建任意恶意代码。

修复或规避建议:在动态构建代码语句前对输入数据进行验证,确保仅能用于构建允许执行的代码。

示例 1: 不规范用法

```
import javax.script.* ;
public class Example{
    public void exampleFun(String code){ //code 为用户输入数据
        ScriptEngineManager manager = new ScriptEngineManager();
        ScriptEngine engine = manager.getEngineByName("javascript");
        engine.eval(code); //code 中可能含有恶意的可执行代码
        ...
    }
}
```

示例 2: 规范用法之一

```
import javax.script.* ;
public class Example{
    public void exampleFun(String code){ //code 为用户输入数据
        ScriptEngineManager manager = new ScriptEngineManager();
        ScriptEngine engine = manager.getEngineByName("javascript");
        String regex = "^document\\.write(|ln)\\([A-Za-z0-9]+\\)\\;$ "; //正则表达式
        if(code != null && code.matches(regex)){ //验证 code 中的内容是否匹配正则表达式
            engine.eval(code);
            ...
        }
    }
}
```

6.2.3.6 进程控制

漏洞描述:使用未经验证的输入数据作为动态加载的库的标识符。

漏洞风险:使用未经验证的输入数据作为动态加载的库的标识符使攻击者有机会加载恶意的代码库。

修复或规避建议:在动态加载库前对输入数据进行验证,确保输入数据仅能用于加载允许加载的代码库。

示例 1：不规范用法

```
public class Example{
    void loadLib(String libraryName){ //libraryName 为用户输入数据
        String path = "C:\\\" + libraryName;
        //使用未经验证的输入数据作为动态加载的库可能导致加载恶意代码库
        System.load(path);
        ...
    }
}
```

示例 2：规范用法之一

```
public class Example{
    boolean verification(String){ //验证库是否合法
        ...
    }
    void loadLib(String libraryName){ //libraryName 为用户输入数据
        String path = "C:\\\";
        //判断 libraryName 是否合法
        if(verification(libraryName)){
            path += libraryName;
            System.load(path);
            ...
        }
    }
}
```

6.2.3.7 信息通过错误消息泄露

漏洞描述:软件呈现给用户的错误消息中包括与环境、用户或相关数据有关的敏感信息。
漏洞风险:对攻击者而言,敏感信息可能本身就是有价值的信息,或者有助于开展进一步的攻击。
修复或规避建议:确保错误消息中仅含有对目标受众有用的少量细节。

示例 1：不规范用法

```
import java.sql.*;
import javax.servlet.ServletException;
public class Example extends Servlet{
    private String dbUrl; //数据库的端口和名称
    private String username;
    private String password;
    public void exampleFun(HttpServletRequest request){
        ...
        Connection con = null;
        //连接数据库有可能抛出异常,页面将跳转至默认异常页面,该页面会包含异常的相关信息
        con = DriverManager.getConnection(dbUrl, username, password);
        ...
    }
    ...
}
```

示例 2：规范用法之一

```
import java.sql.*;
import javax.servlet.ServletException;
public class Example extends Servlet{
    private String dbUrl; //数据库的端口和名称
    private String username;
    private String password;
    public void exampleFun(HttpServletRequest request){
        ...
        Connection con = null;
        try{
            con = DriverManager.getConnection(dbUrl, username, password); //连接数据库
        }
        catch (Exception e){ //捕获该异常,不将该异常的相关信息呈现给用户
            String msg = "Sorry! We will fix the problem soon!";
            request.getSession().setAttribute(msg, errorMsg);
            ...
        }
        ...
    }
    ...
}
```

示例 3：规范用法之一

```
<!--在配置文件 web.xml 中添加以下内容-->

<error-page>
    <exception-type>java.lang.Throwable</exception-type> <!--要捕捉的异常类型-->
    <location>/exception_handler.jsp</location><!--出现异常时跳转的页面,该页面不包含异常相关的信息-->
</error-page>
```

6.2.3.8 信息通过服务器日志文件泄露

漏洞描述：将敏感信息写入服务器日志文件。

漏洞风险：攻击者有机会通过访问日志文件读取敏感信息。

修复或规避建议：慎重考虑写入日志文件信息的隐私性。不要把敏感信息写入日志文件。

示例 1：不规范用法

```
public class Example{
    public void writeLog(String msg){ //将 msg 写入日志文件中
        ...
    }
    private boolean isTrue(String username, String password){ //判断用户名口令是否正确
        ...
    }
}
```

```

public void doGet(HttpServletRequest request, HttpServletResponse response)
    Throws ServletException, IOException{
    String username = request.getParameter("username");
    String password = request.getParameter("password");
    if(isTrue(username, password)){ //用户名和口令正确时调用 writeLog()写入日志
        String msg = username + " and " + password + " correct!"; //msg 中包含了敏感信息
        writeLog(msg);
        ...
    }
}
}

```

示例 2：规范用法之一

```

public class Example{
    public void writeLog(String msg){ //将 msg 写入日志文件中
        ...
    }
    private boolean isTrue(String username, String password){ //判断用户名口令是否正确
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if(isTrue(username, password)){
            String msg = "Login success!"; //msg 中不写入敏感信息
            writeLog(msg);
            ...
        }
    }
}
}

```

6.2.3.9 信息通过调试日志文件泄露

漏洞描述：应用程序没有充分限制对用于调试的日志文件的访问。

漏洞风险：调试日志文件通常包含应用程序的敏感信息，攻击者有机会通过访问调试日志文件读取敏感信息。

修复或规避建议：在产品发布之前移除产生日志文件的代码。

示例 1：不规范用法

```

<!--以下为 log4j.properties 配置文件的部分内容-->
<!--设置日志级别为 DEBUG-->
log4j.rootLogger=DEBUG,LOG1
<!--设置日志输出为文件-->
log4j.appender.LOG1=org.apache.log4j.FileAppender
<!--设置日志输出的路径-->
log4j.appender.LOG1.File=c:/Data.log
<!--设置日志内容样式-->
log4j.appender.LOG1.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p [%t] %c %x - %m%n

-----

//以下为 DataStore.java 的部分内容
import org.apache.log4j.Logger;
public class DataStore {
    private static Logger logger = Logger.getLogger(DataStore.class);
    private String msg;
    ...
    public static void main(String[] args){
        logger.debug(msg); //输出 msg 到日志文件中
        ...
    }
}

```

示例 2：规范用法之一

```

<!--以下为 log4j.properties 配置文件的部分内容-->
<!--设置日志级别为 INFO,则 INFO 级别以下的日志将不会得到输出-->
log4j.rootLogger=INFO, LOG1
<!--设置日志输出为文件-->
log4j.appender.LOG1=org.apache.log4j.FileAppender
<!--设置日志输出的路径-->
log4j.appender.LOG1.File=c:/Data.log
<!--设置日志内容样式-->
log4j.appender.LOG1.layout.ConversionPattern=%d{yyyy-MM-dd HH:mm:ss} %-5p [%t] %c %x - %m%n

-----

//以下为 DataStore.java 的部分内容
import org.apache.log4j.Logger;
public class DataStore {
    private static Logger logger = Logger.getLogger(DataStore.class);
    private String msg;
    ...
    public static void main(String[] args){
        logger.debug(msg); //由于日志级别是 INFO,DEBUG 级别的日志将不会得到输出
        ...
    }
}

```

6.2.3.10 信息通过持久 cookie 泄露

漏洞描述:将敏感信息存储到持久 cookie 中。

漏洞风险:持久 cookie 是被存储在浏览器所在的硬盘驱动器上的 cookie,攻击者有机会通过访问硬盘上的 cookie 文件读取敏感信息。

修复或规避建议:不要在持久 cookie 中保存敏感信息。

示例 1: 不规范用法

```
public class Example{
    public void doPost(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String cookieName="Sender";
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        ...
        //持久 cookie 中保存用户名和口令,用于 14 天内自动登录
        Cookie cookieUsername = new Cookie(cookieName + "Name;", username);
        cookieUsername.setMaxAge(14 * 24 * 60 * 60); //存活期为 14 天
        response.addCookie(cookieUsername);
        Cookie cookiePassword = new Cookie(cookieName + "Password;", password);
        cookiePassword.setMaxAge(14 * 24 * 60 * 60); //存活期为 14 天
        response.addCookie(cookiePassword);
        ...
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    public void doPost(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String cookieName="Sender";
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        ...
        Cookie cookieUsername = new Cookie(cookieName + "Name", username);
        cookieUsername.setMaxAge(14 * 24 * 60 * 60); //存活期为 14 天
        response.addCookie(cookieUsername);
        //由于持久 cookie 存储于客户端,容易被攻击,所以不应将口令保存到持久 cookie 中
        /*
        Cookie cookiePassword = new Cookie(cookieName + "Password", password);
        cookiePassword.setMaxAge(14 * 24 * 60 * 60); //存活期为 14 天
        response.addCookie(cookiePassword);
        */
        ...
    }
}
```

6.2.3.11 未检查的输入作为循环条件

漏洞描述:软件没有对被用于循环条件的输入进行适当的检查。

漏洞风险:攻击者可以让软件循环过多而使软件拒绝服务。

修复或规避建议:规定循环次数的上限,在将用户输入的数据用于循环条件前验证用户输入的数据是否超过上限。

示例 1: 不规范用法

```
public class Example{
    public void exampleFun(int count){ //count 为用户输入数据
        int i;
        if(count > 0){ //如果 count 过大可能使软件拒绝服务
            for(i = 0;i < count;i++){
                ...
            }
        }
        ...
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    private int MAX_COUNT = 1000; //定义最大循环次数
    public void exampleFun(int count){ //count 为用户输入数据
        int i;
        if(count > 0 && count <= MAX_COUNT){ //判断 count 是否在限制次数内
            for(i = 0;i < count;i++){
                ...
            }
        }
        ...
    }
}
```

6.2.3.12 XPath 注入

漏洞描述:使用未经验证的输入动态地构造一个 XPath 表达式,来从 XML 数据库中检索数据。

漏洞风险:攻击者可以构造恶意的 XPath 表达式对 XML 数据库进行越权操作。

修复或规避建议:使用参数化的 XPath 查询(例如使用 XQuery),这将有助于确保数据平面和控制平面相分离。正确验证用户输入,在适当的情况下拒绝数据、过滤数据或对数据进行转义。

示例 1: 不规范用法

```
import javax.xml.xpath.*;
public class Example{
    public void doPost(HttpServletRequest req, HttpServletResponse rsp)
        throws ServletException, IOException{
        XQDataSource ds = new SaxonXQDataSource();
        XQConnection conn = ds.getConnection();
        XQExpression expression = conn.createExpression();
        String author = req.getParameter("author");
        //author 中可能包含恶意表达式
        String query = "for $ x in doc(/\"books.xml/\")//bookstore//book "
            + "where $ x/author=/" + author + "/" "
            + "return $ x/title";
        XQResultSequence res = expression.executeQuery(query);
        ...
    }
}
```

示例 2: 规范用法之一

```
import javax.xml.xpath.*;
public class Example{
    public void doPost(HttpServletRequest req, HttpServletResponse rsp)
        throws ServletException, IOException{
        XQDataSource ds = new SaxonXQDataSource();
        XQConnection conn = ds.getConnection();
        XQExpression xqexp = conn.createExpression();
        String author = req.getParameter("author");
        XQItemType strType = conn.CreateAtomicType(XQItemType.XQBASETYPE_STRING);
        if(author != null){
            String query = "declare variable $ author as xs:string external;" +
                "for $ x in doc(/\"books.xml/\")//bookstore//book " +
                "where $ x/author=$ author" + "return $ x/title";
            xqexp.bindString(new QName("author"), author, strType);
            XQResultSequence res = xqexp.executeQuery(query);
            ...
        }
        ...
    }
}
```

示例 3：规范用法之一

```
import javax.xml.xpath.*;
public class Example{
    private String regex = "[A-Za-z\\.]+ $"; //正则表达式
    ...
    public void doPost(HttpServletRequest req, HttpServletResponse rsp)
        Throws ServletException, IOException{
        XQDataSource ds = new SaxonXQDataSource();
        XQConnection conn = ds.getConnection();
        XQExpression expression = conn.createExpression();
        String author = req.getParameter("author");

        if(author != null && author.matches(regex)){ //如果 author 匹配正则表达式则允许用它构建查询语句
            String query = "for $ x in doc(\"books.xml/\")//bookstore//book "
                + "where $ x//author=/" + author + "/" "
                + "return $ x/title";
            XQResultSequence res = expression.executeQuery(query);
            ...
        }
        ...
    }
}
```

6.2.4 处理程序错误

6.2.4.1 未限制危险类型文件的上传

漏洞描述：软件没有限制允许用户上传的文件类型。

漏洞风险：攻击者可上传具有危险类型的文件，这些文件可能在产品的环境中自动被处理。

修复或规避建议：限制允许用户上传的文件类型。

示例 1：不规范用法

```
import org.apache.commons.fileupload.servlet.ServletFileUpload;
import org.apache.commons.fileupload.disk.DiskFileItemFactory;
public class Example{
    public void exampleFun (HttpServletRequest request) {
        DiskFileItemFactory factory = new DiskFileItemFactory();
        ServletFileUpload upload = new ServletFileUpload(factory);
        List<FileItem> items = upload.parseRequest(request);
        Iterator<FileItem> iter = items.iterator();
        while (iter.hasNext()) {
            FileItem item = iter.next();
            //对文件的相关操作,但没有判断文件类型
            ...
        }
    }
}
```

示例 2：规范用法之一

```
import org.apache.commons.fileupload.servlet.ServletFileUpload;
import org.apache.commons.fileupload.disk.DiskFileItemFactory;

public class Example{
    private String regex = "-[gif|jpg|bmp]$ "; //正则表达式
    public void exampleFun (HttpServletRequest request) {
        DiskFileItemFactory factory = new DiskFileItemFactory();
        ServletFileUpload upload = new ServletFileUpload(factory);
        List<FileItem> items = upload.parseRequest(request);
        Iterator<FileItem> iter = items.iterator();
        while (iter.hasNext()) {
            FileItem item = iter.next();
            String fileName = item.getName();
            //将 fileName 中的所有 0x00 字符替换成"_"，规避 00 截断上传漏洞
            fileName = fileName.replaceAll("\\p{Cntrl}", "_");
            String fileEnd = fileName.substring(fileName.lastIndexOf(".") + 1).toLowerCase();
            if(fileEnd.matches(regex)){ //验证文件类型
                //对文件的相关操作
                ...
            }
        }
    }
}
```

6.2.5 不充分的封装

6.2.5.1 可序列化的类包含敏感数据

漏洞描述:代码包含一个有敏感数据的类,但是类没有显式的拒绝序列化。
 漏洞风险:攻击者可对包含有敏感数据的类进行序列化,从而使用存储在里面的敏感数据。
 修复或规避建议:定义 writeObject()方法去抛出一个异常,显式地拒绝序列化。

示例 1：不规范用法

```
class Staff implements Serializable{ //攻击者可以对此类进行序列化,获得存储在里面的 name 和 password,即使它们是私有的
    private String name;
    private String password;
    ...
}
```

示例 2：规范用法之一

```
class Staff implements Serializable{
    private String name;
    private String password;
    ...
    private void writeObject(ObjectOutputStream out){ //定义 writeObject()方法显式地拒绝序列化
        throw new Exception("Cannot be serialized!");
    }
}
```

6.2.5.2 违反信任边界

漏洞描述:让数据从不受信任的一边移到受信任的一边却未经验证。

漏洞风险:程序员更容易错误地相信那些未被验证的数据,导致未经验证的数据被攻击者利用。

信任边界可以被认为是画在程序中的一条线。线的一侧是不受信任数据;另一侧,我们认为数据是可以被信任的。验证逻辑旨在让数据安全地穿过信任边界——从不受信任的一边移到受信任的一边。

修复或规避建议:增加验证逻辑让数据安全地穿过信任边界——从不受信任的一边移到受信任的一边。

示例 1: 不规范用法

```
public class Example{
    protected void doPost (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException{
        String username = request.getParameter("username");
        HttpSession session = request.getSession(true);
        session.setAttribute("username", username); //没有验证 username 是否可信
        ...
    }
}
```

示例 2: 规范用法之一

```
public class Example{
    protected void doPost (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException{
        String username = request.getParameter("username");
        HttpSession session = request.getSession(true);
        //不把不受信任的数据放到 session 中
        //session.setAttribute("username", username);
        ...
    }
}
```

6.2.6 安全功能

6.2.6.1 明文存储口令

漏洞描述:将口令明文存储。

漏洞风险:明文存储口令会降低攻击的难度,攻击者若获得访问口令存储文件的权限,便可轻易获取口令。

修复或规避建议:尽量避免在容易受攻击的地方存储口令。如果需要,考虑存储口令的加密散列,以替代明文口令存储。

示例 1: 不规范用法

```
public class Example{
    public void storePassword(Connection con, String id, String password){
        PreparedStatement ps = con.prepareStatement("UPDATE user SET password = ? WHERE id = ?");
        ps.setString(1, password); //明文存储 password
        ps.setString(2, id);
        ResultSet results = ps.executeQuery();
        ...
    }
}
```

示例 2: 规范用法之一

```
import java.security.MessageDigest;
public class Example{
    public String encrypt(String str){ //加密函数,返回加密后的字符串
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        ...
    }
    public void storePassword(Connection con, String id, String password){
        PreparedStatement ps = con.prepareStatement("UPDATE user SET password = ? WHERE id = ?");
        String psw = encrypt(password); //加密 password
        ps.setString(1, psw); //存储加密后的 psw
        ps.setString(2, id);
        ResultSet results = ps.executeQuery();
        ...
    }
}
```

6.2.6.2 存储可恢复的口令

漏洞描述:采用双向可逆的加密算法加密口令并将口令存储在外部文件或数据库中。

漏洞风险:攻击者有机会通过解密算法对口令进行解密。

修复或规避建议:当业务不需要取已存储的口令进行还原时,使用单向加密算法对口令进行加密并存储。

示例 1: 不规范用法

```
import javax.crypto.Cipher;
public class Example{
    public String encrypt(String str){
        Cipher cipher = Cipher.getInstance("AES"); //使用双向可逆的 AES 算法
        ...
    }
    public void storePassword(Connection con, String id, String password){
        PreparedStatement ps = con.prepareStatement("UPDATE user SET password = ? WHERE id = ?");
        String psw = encrypt(password); //加密 password
        ps.setString(1, psw); //存储加密后的 psw
        ps.setString(2, id);
        ResultSet results = ps.executeQuery();
        ...
    }
}
```

示例 2：规范用法之一

```
import java.security.MessageDigest;
public class Example{
    public String encrypt(String str){
        MessageDigest md=MessageDigest.getInstance("SHA-256"); //使用单向不可逆的 SHA256 加密算法
        ...
    }
    public void storePassword(Connection con, String id, String password){
        PreparedStatement ps = con.prepareStatement("UPDATE user SET password = ? WHERE id = ?");
        String psw = encrypt(password); //加密 password
        ps.setString(1, psw); //存储加密后的 psw
        ps.setString(2, id);
        ResultSet results = ps.executeQuery();
        ...
    }
}
```

6.2.6.3 口令硬编码

漏洞描述：程序代码(包括注释)中包含硬编码口令。

漏洞风险：攻击者可以通过反编译或直接读取二进制代码的方式获取硬编码口令。

修复或规避建议：使用单向加密算法对口令进行加密并存储在外部文件或数据库中。

示例 1：不规范用法

```
public class Example{
    public boolean verifyPassword(String password){ //判断口令是否正确
        boolean flag = false;
        if ("021sdg65df4845".equals(password)){ //程序代码中包含硬编码口令
            flag = true;
        }
        return flag;
    }
}
```

示例 2：规范用法之一

```
import java.security.MessageDigest;
public class Example{
    public String encrypt(String str){ //加密函数,返回加密后的字符串
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        ...
    }
    private String getPassword(void){ //从数据库中获得存储的口令
        ...
    }
    public boolean verifyPassword(String password){ //判断口令是否正确
        boolean flag = false;
```

```

        String dbPassword = getPassword();
        String encryptedPassword = encrypt(password); //加密用户输入的 password
        //将加密后得到的 encryptedPassword 和数据库中存储的 dbPassword 进行比较
        if (encryptedPassword != null && encryptedPassword.equals(dbPassword)){
            flag = true;
        }
        return flag;
    }
}

```

6.2.6.4 依赖 referer 字段进行身份鉴别

漏洞描述:依赖 HTTP 请求的 referer 字段进行身份鉴别。

漏洞风险:攻击者可通过修改 HTTP 请求的 referer 字段冒用其他用户的身份。

修复或规避建议:通过用户名口令、数字证书等其他手段对用户身份进行验证。

示例 1: 不规范用法

```

public class Example{
    public boolean exampleFun(HttpServletRequest request){
        String referer = request.getHeader("referer");
        String trustedReferer = "http://www.example.com/";
        if(referer.equals(trustedReferer)){ //依赖 referer 字段进行身份鉴别
            ...
        }
        else{
            ...
        }
    }
}

```

示例 2: 规范用法之一

```

public class Example{
    private boolean userExists(String username, String password){ //判断用户名口令是否正确
        ...
    }
    public boolean exampleFun(HttpServletRequest request){
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username != null && password != null) {
            //通过用户名口令进行身份鉴别
            if (userExists(username, password)) {
                ...
            }
        }
        else{
            ...
        }
    }
}

```

6.2.6.5 Cookie 中的敏感信息明文存储

漏洞描述:敏感信息在 cookie 中明文存储。

漏洞风险:攻击者有机会通过读取 cookie 获取敏感信息。

修复或规避建议:将需要存储到 cookie 中的敏感信息先加密后再存储。

示例 1: 不规范用法

```
public class Example{
    public void exampleFun(String address){ //address 为用户的敏感信息
        //在 cookie 中明文存储 address
        Cookie cookie = new Cookie("Address", address);
        ...
    }
}
```

示例 2: 规范用法之一

```
import javax.crypto.Cipher;
public class Example{
    public String encrypt(String str){ //加密函数,返回加密后的字符串
        Cipher cipher = Cipher.getInstance("AES");
        ...
    }
    public void exampleFun(String address){ //address 为用户的敏感信息
        String addr = encrypt(address); //加密 address
        Cookie cookie = new Cookie("Address", addr); //将加密得到的 addr 存储到 cookie 中
        ...
    }
}
```

6.2.6.6 敏感信息明文传输

漏洞描述:敏感信息在传输过程中未进行加密。

漏洞风险:攻击者有机会通过在传输过程中截取或复制报文获取敏感信息。

修复或规避建议:发送敏感信息前对敏感信息进行加密或采用加密通道传输敏感信息。

示例 1: 不规范用法

```
public class Example{
    public void sendMessage(String str){ //将传进来的字符串发送出去
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException{
        String account = request.getParameter("account");
        sendMessage(account); //传输 account 前没有进行加密
        ...
    }
}
```

示例 2：规范用法之一

```
import javax.crypto.Cipher;
public class Example{
    public void encrypt(String str){ //加密函数,返回加密后的字符串
        Cipher cipher = Cipher.getInstance("RSA/ECB/OAEPWITHSHA256AndMGF1Padding");
        ...
    }
    public void sendMessage(String str){ //将传进来的字符串发送出去
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String account = request.getParameter("account");
        String encryptedAccount = encrypt(account); //加密 account
        sendMessage(encryptedAccount); //传输加密后得到的 encryptedAccount
        ...
    }
}
```

6.2.6.7 使用已破解或危险的加密算法

漏洞描述：软件采用已破解或自定义的非标准加密算法。

漏洞风险：使用已破解或非标准的加密算法是很危险的，因为攻击者有可能会破解算法，从而窃取所保护的数据。

修复或规避建议：采用目前加密领域中加密强度较高的标准加密算法。

示例 1：不规范用法

```
import javax.crypto.SecretKey;
import javax.crypto.KeyGenerator;
import javax.crypto.Cipher;
public class Example{
    public void encrypt(String str){
        Cipher cipher = Cipher.getInstance("DES"); //使用加密强度较低的 DES 算法
        SecretKey secretKey = KeyGenerator.getInstance("DES").generateKey(); //定义密钥
        cipher.init(Cipher.ENCRYPT_MODE, secretKey); //使用加密模式初始化密钥
        byte[] result = cipher.doFinal(str.getBytes()); //加密
        ...
    }
}
```

示例 2：规范用法之一

```
import javax.crypto.SecretKey;
import javax.crypto.KeyGenerator;
import javax.crypto.Cipher;
public class Example{
    public void encrypt(String str){
        Cipher cipher = Cipher.getInstance("AES"); //使用加密强度更高的 AES 算法
        SecretKey secretKey = KeyGenerator.getInstance("AES").generateKey(); //定义密钥
        cipher.init(Cipher.ENCRYPT_MODE, secretKey); //使用加密模式初始化密钥
        byte[] result = cipher.doFinal(str.getBytes()); //加密
        ...
    }
}
```

6.2.6.8 可逆的散列算法

漏洞描述：软件采用的散列算法具有可逆算法，该算法可利用生成的散列值确定原始输入，或者可以找到一个输入产生相同的散列值。

漏洞风险：攻击者可通过逆向算法获取原始输入或产生相同散列值的输入，进而绕过依赖散列算法的安全认证。

修复或规避建议：采用当前公认的不可逆标准散列算法。

示例 1：不规范用法

```
import java.security.MessageDigest;
public class Example{
    public void encrypt(String str){
        //使用 SHA-1 散列加密算法
        MessageDigest md = MessageDigest.getInstance("SHA-1");
        ...
    }
}
```

示例 2：规范用法之一

```
import java.security.MessageDigest;
public class Example{
    public void encrypt(String str){
        //使用 SHA-256 标准散列加密算法
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        ...
    }
}
```

6.2.6.9 密码分组链接模式未使用随机初始化矢量

漏洞描述:密码分组链接模式使用的初始化向量不是一个随机数。

漏洞风险:攻击者有机会通过字典式攻击读取加密的数据。

修复或规避建议:密码分组链接模式使用随机的初始化向量。

示例 1: 不规范用法

```
import javax.crypto.SecretKey;
import javax.crypto.KeyGenerator;
import javax.crypto.Cipher;
public class Example{
    public static byte[] encryptData(String plaintext) {
        byte[] iv = { //设置初始化向量为固定的值
            0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00
        };
        KeyGenerator kg = KeyGenerator.getInstance("AES");
        SecretKey key = kg.generateKey();
        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        IvParameterSpec ips = new IvParameterSpec(iv); //使用固定的初始化向量
        cipher.init(Cipher.ENCRYPT_MODE, key, ips);
        return cipher.doFinal(plaintext.getBytes());
    }
}
```

示例 2: 规范用法之一

```
import javax.crypto.SecretKey;
import javax.crypto.KeyGenerator;
import javax.crypto.Cipher;
import java.security.*;
public class Example{
    public static byte[] encryptData(String plaintext) {
        byte[] iv = new byte[8];
        Random random = new SecureRandom(); //基于加密的伪随机数生成器 CSPRNG
        random.nextBytes(iv); //随机生成初始化向量
        KeyGenerator kg = KeyGenerator.getInstance("AES");
        SecretKey key = kg.generateKey();
        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        IvParameterSpec ips = new IvParameterSpec(iv); //使用随机的初始化向量
        cipher.init(Cipher.ENCRYPT_MODE, key, ips);
        return cipher.doFinal(plaintext.getBytes());
    }
}
```

6.2.6.10 不充分的随机数

漏洞描述:软件在安全相关代码中依赖不充分的随机数。

漏洞风险:攻击者可以预测将生成的随机数,绕过依赖随机数的安全保护。

修复或规避建议:使用目前被业界专家认为较强的经过良好审核的加密 PRNG 算法,初始化随机数生成器时使用具有足够长度且不固定的种子。

示例 1: 不规范用法

```
public class Example{
    public void exampleFun(){
        byte[] iv = new byte[8];
        Random random = new Random(); //不充分的伪随机数生成器
        random.nextBytes(iv);
        ...
    }
}
```

示例 2: 规范用法之一

```
import java.security.*;
public class Example{
    public void exampleFun(){
        byte[] iv = new byte[8];
        Random random = new SecureRandom(); //基于加密的伪随机数生成器 CSPRNG
        random.nextBytes(iv);
        ...
    }
}
```

6.2.6.11 安全关键的行为依赖反向域名解析

漏洞描述:通过反向域名解析获取 IP 地址的域名,然后依赖域名对主机进行身份鉴别。

漏洞风险:攻击者可通过 DNS 欺骗修改 IP 地址与域名的对应关系,从而绕过依赖域名的主机身份鉴别。

修复或规避建议:通过用户名口令、数字证书等其他手段对主机身份进行鉴别。

示例 1: 不规范用法

```
import java.net.*;
public class Example{
    private boolean trusted;
    ...
    public void getTrust(HttpServletRequest request){
        String ip = request.getRemoteAddr();
        InetAddress address = InetAddress.getByName(ip);
        //攻击者可通过 DNS 欺骗绕过依赖域名的主机身份鉴别
        if (address.getCanonicalHostName().endsWith("trustme.com")) {
            trusted = true;
        }
    }
}
```

示例 2：规范用法之一

```
import java.net.*;
public class Example{
    private boolean trusted;
    private boolean userExists(String username, String password){ //判断用户名口令是否正确
        ...
    }
    ...
    public void getTrust(HttpServletRequest request){
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username != null && password != null) {
            //通过用户名口令进行主机身份鉴别
            if (userExists(username, password)) {
                trusted = true;
            }
        }
    }
}
```

6.2.6.12 关键参数篡改

漏洞描述：软件未经验证使用可能被篡改的关键参数（如资产数据等）。

漏洞风险：某些数据如页面隐藏域字段、Cookie、URL 等外部渠道传入的关键参数可能被恶意的浏览器插件、代理服务器等方式篡改。若程序获取该类数据后不经验证直接使用，可能造成信息泄露、数据异常、系统故障等安全事件。

修复或规避建议：将关键参数缓存到服务端的会话中，程序使用该数据须通过会话获取。

示例 1：不规范用法

```
class Example{
    public String encode(String str){
        //编码转义函数，用于规避 sql 注入、跨站脚本等攻击
        ...
    }
    public void exampleFun(){
        /* *
         * 本例模拟黑客攻击购物网站
         * */
        String prices = request.getParameter("price"); //price 是页面隐藏域中的商品单价，属于关键参数
        double price = Double.parseDouble(encode(prices));
        String numbers = request.getParameter("number"); //number 是商品数量，不属于关键参数
        int number = Integer.parseInt(encode(numbers));
        //该购物网站依赖页面隐藏域传回的商品单价生成账单，黑客可篡改商品单价实现“0 元”购物
        double amount = price * number;
        ...
    }
}
```

示例 2：规范用法之一

```

class Example{
    public String encode(String str){
        //编码转义函数,用于规避 sql 注入、跨站脚本等攻击
        ...
    }
    public void exampleFun(){
        HashMap<String, double> priceMap = new HashMap(String, double); //商品 ID 与商品单价对应表
        ...
        String productID = request.getParameter("productID");
        double price = priceMap.get(encode(productID));
        String numbers = request.getParameter("number"); //number 是商品数量,不属于关键参数
        int number = Integer.parseInt(encode(numbers));
        //该购物网站依赖服务端缓存的商品单价生成账单,规避商品单价这一关键参数被篡改的风险
        double amount = price * number;
        ...
    }
}

```

6.2.6.13 没有要求使用强口令

漏洞描述:软件没有要求用户使用具有足够复杂度的口令。

漏洞风险:攻击者可以很容易地猜出用户的口令或实施暴力破解攻击。

修复或规避建议:要求用户使用具有足够复杂度的口令。口令强度策略应包含下列属性:

- 最小和最大长度;
- 包含字母、数字和特殊字符;
- 不包含用户名;
- 定期更改口令;
- 不使用用过的口令。

示例 1：不规范用法

```

class Staff{
    ...
    public void storePassword(String psw){ //存储用户输入的口令
        ...
    }
    public void doGet(HttpServletRequest req, HttpServletResponse rsp)
        Throws ServletException, IOException{
        String password = req.getParameter("password");
        storePassword(password); //口令强度可能不足
        ...
    }
}

```

示例 2：规范用法之一

```

class Staff{
    private String username; //用户名
    private String[] passwordlist; //口令表
    private String createPasswordDate; //口令创建的时间
    private int passwordOutdateDays; //口令过期天数(可配置)
    private boolean userExists(String username, String password){ //判断用户名口令是否正确
        ...
    }
    public void storePassword(String password){ //存储用户输入的口令
        ...
    }
    public boolean checkLength(String password){ //验证口令长度
        ...
    }
    public boolean checkMode(String password){
        //口令是否包含字母,数字和特殊字符,缺少其中一种或以上表示口令强度弱
        ...
    }
    public boolean checkExcludeName(String password){ //口令是否不包含用户名
        ...
    }
    public boolean checkTime(String password){ //通过比较当前时间和口令创建时间判断口令是否过期
        ...
    }
    public boolean checkIsUsed(String password){ //通过查询口令表判断口令是否曾经使用过
        ...
    }
    ...
    public boolean checkPasswordLevel(String password){ //口令强度检测
        if(! checkLength(password)){
            System.out.println("口令长度不符合要求!");
            return false;
        }
        if(! checkMode(password)){
            System.out.println("口令组合等级弱!");
            return false;
        }
        if(! checkExcludeName(password)){
            System.out.println("口令包含用户名!");
            return false;
        }
        if(! checkIsUsed(password)){
            System.out.println("口令曾经使用过!");

```

```

        return false;
    }
    ...
    return true;
}

public void doGet(HttpServletRequest req, HttpServletResponse rsp)
    Throws ServletException, IOException{
    String password = req.getParameter("password");
    if(checkPasswordLevel(password)){ //检测 password 的口令强度
        storePassword(password);
    }
    else{
        ...
    }
}

//用户登录时自动判断口令使用期限并提示用户更新口令
public void checkUser(String username, String password){
    if(userExists(username, password)){ //身份鉴别
        if(! checkTime(password)){ //检测 password 是否过期
            //提示用户口令已过期,建议其修改口令
            ...
        }
    }
    ...
}
}

```

6.2.6.14 没有对口令域进行掩饰

漏洞描述:用户输入口令时没有对口令域进行掩饰。

漏洞风险:增加了攻击者通过观察屏幕获取到口令的可能性。

修复或规避建议:用户输入口令时对口令域进行掩饰。通常,用户输入的每一个字符都应该以星号形式回显。

示例 1: 不规范用法

```

import javax.swing.* ;

public class Example{
    public void exampleFun(){
        //口令域使用明文输入的 JTextField 控件
        JTextField passwordfield = new JTextField();
        ...
    }
}

```

示例 2：规范用法之一

```
import javax.swing.*;
public class Example{
    public void exampleFun(){
        //口令域使用可以掩饰口令输入的 JPasswordField 控件
        JPasswordField passwordfield = new JPasswordField();
        passwordfield.setEchoChar('*'); //设置回显的符号为 '*'
        ...
    }
}
```

6.2.6.15 依赖未经验证和完整性检查的 cookie

漏洞描述：应用程序当执行安全性非常重要的操作时，依赖 Cookies 的存在或它的值，而又没有正确地保证这些设置对相关的用户是有效的。

漏洞风险：攻击者可以很容易的在浏览器中或浏览器外的客户端代码修改 cookie。依赖未经验证和完整性检查的 cookie 可以允许攻击者绕过验证，执行像 SQL 注入这样的注入攻击和跨站脚本攻击或者以不期待的方式修改输入。

修复或规避建议：在做安全相关的决定时，依赖服务器端存储的数据，避免依赖客户端传过来的 Cookie 数据。

示例 1：不规范用法

```
public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        Cookie[] cookies = request.getCookies();
        if(cookies != null){
            for(int i=0;i<cookies.length;i++){
                if("isAdmin".equals(cookies[i].getName())){
                    String isAdmin = cookies[i].getValue();
                    //用 cookie 存储的 isAdmin 字段来判断用户是否为系统管理员
                    if("true".equals(isAdmin)){
                        ...
                    }
                    ...
                    break;
                }
            }
        }
        ...
    }
}
```

示例 2：规范用法之一

```

public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        HttpSession session = request.getSession();
        String isAdmin = session.getAttribute("isAdmin");
        if("true".equals(isAdmin)){ //根据 session 判断用户是否为系统管理员
            ...
        }
        ...
    }
}

```

6.2.6.16 通过用户控制的 SQL 关键字绕过授权

漏洞描述：软件使用的数据库表其中包括了某个用户不应有权访问的记录，但该软件执行的一个 SQL 语句中的关键字却可以受该用户控制。

漏洞风险：如果用户可以将关键字设置为任何值，那么该用户就可以修改该关键字指向未经授权的记录。

修复或规避建议：对用户输入的关键字进行验证，确保其只能访问该用户有权访问的记录。

示例 1：不规范用法

```

import java.sql.*;
public class Example{
    public void sqlQuery(Connection con, String staffID){ //staffID 为用户输入数据
        PreparedStatement ps = con.prepareStatement("SELECT * FROM userData WHERE staffID = ?");
        ps.setString(1, staffID); //用户输入的 staffID 可能不在其访问权限内
        ...
    }
}

```

示例 2：规范用法之一

```

import java.sql.*;
public class Example{
    private String username;
    private String findDepartID(String); //根据用户名返回其所在部门的部门 ID
    {
        ...
    }
    public void sqlQuery(Connection con, String staffID){ //staffID 为用户输入数据
        //查询语句加上部门 ID 的查询条件
        PreparedStatement ps = con.prepareStatement("SELECT * FROM employee WHERE staffID = ?"
            + "and DepartID = ?");
        ps.setString(1, staffID);
        ps.setString(2, findDepartID(username));
        ...
    }
}

```

6.2.6.17 HTTPS 会话中的敏感 cookie 没有设置安全属性

漏洞描述:HTTPS 会话中的敏感 cookie 没有设置安全属性。

漏洞风险:若没有设置安全属性,敏感 cookie 可通过 HTTP 会话以明文形式发送,可能敏感 cookie 被攻击者窃取。

修复或规避建议:设置 HTTPS 会话中的敏感 cookie 的安全属性。

示例 1: 不规范用法

```
import javax.crypto.Cipher;
public class Example{
    private String encrypt(String plaintext){ //加密函数,返回加密后的字符串
        Cipher cipher = Cipher.getInstance("AES");
        ...
    }
    public void doPost(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String userID = request.getParameter("userID");
        String id = encrypt(userID); //加密敏感信息 userID
        Cookie cookieID = new Cookie("userID", id);
        response.addCookie(cookieID); //没有设置 cookieID 的 secure 属性
        ...
    }
}
```

示例 2: 规范用法之一

```
import javax.crypto.Cipher;
public class Example{
    private String encrypt(String plaintext){ //加密函数,返回加密后的字符串
        Cipher cipher = Cipher.getInstance("AES");
        ...
    }
    public void doPost(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String userID = request.getParameter("userID");
        String id = encrypt(userID); //加密敏感信息 userID
        Cookie cookieID = new Cookie("userID", id);
        cookieID.setSecure(true); //设置 cookieID 的 secure 属性为 true
        response.addCookie(cookieID);
        ...
    }
}
```

6.2.6.18 未使用盐值计算散列值

漏洞描述:软件针对口令等不应可逆的输入使用了单向加密散列,但未使用盐值。

漏洞风险:如果未使用盐值计算散列值,攻击者就更容易利用彩虹表等字典式攻击技术破解口令。

修复或规避建议:使用盐值计算散列值,增加攻击者破解口令的难度。

示例 1: 不规范用法

```
import java.security.spec.*;
import java.security.MessageDigest;
public class Example{
    private String encrypt(String password, KeySpec key){ //加密函数,返回加密后的字符串
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        ...
    }
    public void storePassword(String password){
        KeySpec keyspec = new PBEKeySpec(password.toCharArray()); //生成密钥
        //仅使用单向加密,还是容易被攻击者用彩虹表等方式破解口令
        String encryptedPassword = encrypt(password, key);
        //将 encryptedPassword 存放到数据库
        ...
    }
}
```

示例 2: 规范用法之一

```
import java.security.spec.*;
import java.security.MessageDigest;
import javax.crypto.Cipher;
public class Example{
    private String encrypt(String password, KeySpec key){ //加密函数,返回加密后的字符串
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        ...
    }
    private void sendSalt(String salt){ //用 AES 算法加密盐值后传输到另一台服务器上
        Cipher cipher = Cipher.getInstance("AES");
        ...
    }
    private int iterationCount = 1000; //迭代次数
    private int length = 32; //盐值长度
    public void storePassword(String password){
        byte[] salt = new byte[length];
        Random random = new SecureRandom();
        random.nextBytes(salt); //随机生成盐值
        //使用盐值生成密钥
        KeySpec key = new PBEKeySpec(password.toCharArray(), salt, iterationCount);
        String encryptedPassword = encrypt(password, key); //增加攻击者破解口令的难度
        //将 encryptedPassword 存入数据库
        ...
        //在另一台服务器的数据库中存放 salt 值
        sendSalt(salt);
        ...
    }
}
```

6.2.6.19 RSA 算法未使用最优非对称加密填充

漏洞描述:使用 RSA 加密算法时未使用最优非对称加密填充。
漏洞风险:未使用最佳非对称加密填充将降低攻击者解密的难度。
修复或规避建议:使用 RSA 加密算法时使用最优非对称加密填充。
示例 1: 不规范用法

```
import javax.crypto.Cipher;
public class Example{
    public void encrypt(String str){
        Cipher cipher = Cipher.getInstance("RSA/ECB/PKCS1Padding"); //使用 Pkcs1 加密标准
        ...
    }
}
```

示例 2: 规范用法之一

```
import javax.crypto.Cipher;
public class Example{
    public void encrypt(String str){
        //使用最优非对称加密填充(OAEP)加密标准
        Cipher cipher = Cipher.getInstance("RSA/ECB/OAEPWITHSHA256AndMGF1Padding");
        ...
    }
}
```

6.2.7 时间和状态

6.2.7.1 会话固定

漏洞描述:对用户进行身份鉴别并建立一个新的会话时没有让原来的会话失效。
漏洞风险:攻击者可诱使用户在攻击者创建的会话基础上进行身份鉴别,从而窃取用户通过身份鉴别的会话并冒充用户进行恶意操作。
修复或规避建议:对用户进行身份鉴别并建立一个新的会话时让原来的会话失效。
示例 1: 不规范用法

```
public class Example{
    private boolean userExists(String username, String password){ //判断用户名口令是否正确
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username != null && password != null){
            //通过用户名口令进行身份鉴别
            if(userExists(username, password)){
```

```

        //没有建立新的会话并让原来的会话失效
        HttpSession session = request.getSession(true);
        ...
    }
}
}
...
}

```

示例 2：规范用法之一

```

public class Example{
    private boolean userExists(String username, String password){ //判断用户名口令是否正确
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if(username != null && password != null){
            //通过用户名口令进行身份鉴别
            if(userExists(username, password)){
                HttpSession session = request.getSession(false);
                if(session != null){ //如果存在旧的会话
                    session.invalidate(); //清空会话数据
                }
                session = request.getSession(true); //建立一个新会话
                ...
            }
        }
    }
    ...
}

```

6.2.7.2 会话永不过期

漏洞描述：设置会话为永不过期。

漏洞风险：攻击者可以有足够的时间重复使用原有的会话凭据或会话 ID 来进行授权。

修复或规避建议：设置会话过期的日期。

示例 1：不规范用法

```

//配置文件 web.xml
<session-config>
    <session-timeout>-1</session-timeout> //该值为-1 时,session 永不过期
</session-config>

```

示例 2：规范用法之一

```
//修改配置文件 web.xml 中的<session-timeout>值
<session-config>
    <session-timeout>5</session-timeout> //session 失效时间设为 5 分钟
</session-config>
```

示例 3：规范用法之一

```
//在 sessioncreate.jsp 中直接设置 session 超时时间,因为 jsp 中的优先级比 web.xml 中的更高
session.setMaxInactiveInterval(300); //session 失效时间为 300 秒=5 分钟
```

6.2.8 Web 问题

6.2.8.1 跨站脚本

漏洞描述:使用未经验证的输入数据构建 Web 页面。
漏洞风险:攻击者可构建任意 Web 页面,并在页面中植入恶意脚本。
修复或规避建议:在构建 Web 页面前对输入数据进行验证或编码,确保输入数据不影响页面的结构。
示例 1：不规范用法

```
public class Example{
    public void doGet(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String text = request.getParameter("text");
        ServletOutputStream out = response.getOutputStream();
        out.write(text); //text 中可能含有恶意脚本
        out.flush();
        ...
        out.close();
    }
}
```

示例 2：规范用法之一

```
public class Example{
    public void doGet(HttpServletRequest request,HttpServletResponse response)
        Throws ServletException,IOException{
        String regex = "\\w+$"; //正则表达式
        String text = request.getParameter("text");
        ServletOutputStream out = response.getOutputStream();
        if(text != null && text.matches(regex)){ //验证 text 中的内容是否匹配正则表达式
            out.write(text);
            out.flush();
            ...
        }
        out.close();
    }
}
```

示例 3：规范用法之一

```
import org.apache.commons.lang.StringEscapeUtils;
public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String text = request.getParameter("text");
        ServletOutputStream out = response.getOutputStream();
        //使用 escapeHtml 方法对 test 进行编码
        out.write(StringEscapeUtils.escapeHtml(text));
        out.flush();
        ...
        out.close();
    }
}
```

6.2.8.2 跨站请求伪造

漏洞描述：Web 产品没有或者不能充分验证用户所提交的请求是否是攻击者伪造的。

漏洞风险：攻击者可以通过 URL、图像载入、XMLHttpRequest 等方法诱骗客户向 Web 服务器作出无意请求，而该请求却会被当作真实请求来处理。

修复或规避建议：服务器端为每一个表单生成一个不可预测的随机数，将该随机数放置到表单中，并在收到表单后立即验证该随机数。当用户执行危险操作时，向用户界面发送一个单独的确认请求，来确保用户确实想要执行该操作。

示例 1：不规范用法

```
public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String account = request.getParameter("account"); //目标账户
        int money = Integer.parseInt(request.getParameter("money")); //转账金额
        //处理转账请求
        ...
    }
}
```

示例 2：规范用法之一

```
public class Example{
    public boolean sendSure(HttpServletResponse response, String msg){
        //向用户界面发送一个单独的确认请求,用户确认则返回 true
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        HttpSession session = request.getSession();
        String account = request.getParameter("account"); //目标账户
    }
}
```

```

int money = Integer.parseInt(request.getParameter("money")); //转账金额
//secretkey 为创建表单时生成的一个不可预测的随机数
String secretKey = session.getAttribute("secretKey");
String key = request.getParameter("secretkey");
if(key != null && secretKey.equals(key)){ //验证 key
    //向用户界面发送一个单独的确认请求
    String msg = "即将进行转账操作,是否继续?";
    if(sendSure(response, msg)){
        //处理转账请求
        ...
    }
}
}
}
}

```

6.2.8.3 HTTP 响应拆分

漏洞描述:将未经验证的输入数据写入 HTTP 响应的报头。

漏洞风险:攻击者可在输入数据中包含回车换行字符,使 HTTP 响应拆分为两个或多个响应,并构建恶意的 HTTP 响应报文发给共享相同的服务器 TCP 连接的用户。

修复或规避建议:在写入 HTTP 响应的报头前对输入数据进行验证或编码,确保输入数据不包含回车换行字符。

示例 1: 不规范用法

```

public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse rsp)
        Throws ServletException,IOException{
        String type = request.getParameter("content_type");
        response.setHeader("Content-Type",type); //type 中有可能包含回车换行字符("\r\n")
        ...
    }
}

```

示例 2: 规范用法之一

```

public class Example{
    private static int NUM;
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException,IOException{
        //设置白名单
        String[] whitelist = new String[NUM];
        whitelist[0] = "text/html";
        whitelist[1] = "text/plain";
    }
}

```

```

...
String type = request.getParameter("content_type");
int i;
for (i = 0; i < NUM; i++) { //验证 type 是否符合写入 http 响应报头的要求
    if(whitelist[i].equals(type)){
        break;
    }
}
if(i != NUM){
    response.setHeader("Content-Type", type);
    ...
}
}
}

```

6.2.8.4 开放重定向

漏洞描述:使用未经验证的输入数据重定向 URL。

漏洞风险:通过重定向至恶意站点,攻击者可能会成功地发动钓鱼式攻击并窃取用户凭证。由于重定向后的恶意站点显示的 URL 与原来网站的 URL 相同,因此钓鱼攻击就被赋予了一个更值得信赖的外观。

修复或规避建议:在重定向前对输入数据进行验证,确保仅重定向至允许的 URL。或者在重定向至未知站点时向用户发出明确警告。

示例 1: 不规范用法

```

public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        String url = request.getParameter("url");
        response.sendRedirect(url); //未经验证的 url 可能是恶意的
        return;
    }
}

```

示例 2: 规范用法之一

```

public class Example{
    private boolean isTrusted(String url){ //判断一个 url 是否为可信任的
        ...
    }
    public String msgAlert(String msg){ //把 msg 封装成 javascript 警告脚本
        ...
    }
    public void doGet(HttpServletRequest request, HttpServletResponse response)

```

```

        Throws ServletException, IOException{
        String url = request.getParameter("url");
        if(isTrusted(url)){ //判断 url 是否可信任
            response.sendRedirect(url); //重定向
            return;
        }
        else {
            PrintWriter rout = response.getWriter();
            String msg = msgAlert("警告！即将访问未知站点,是否继续?");
            rout.println(msg); //向用户发出警告信息
            ...
        }
    }
}

```

6.2.8.5 依赖外部提供的文件的名称或扩展名

漏洞描述:软件依赖用户上传的文件的名称或扩展名决定软件的行为。

漏洞风险:攻击者可通过提供指定的文件名或扩展名改变软件的行为。

修复或规避建议:在服务器端依赖文件的内容决定软件的行为。

示例 1: 不规范用法

```

import org.apache.commons.fileupload.servlet.ServletFileUpload;
import org.apache.commons.fileupload.disk.DiskFileItemFactory;
public class Example{
    private String regex = "[gif|jpg|bmp]$"; //正则表达式
    public void imageBeauty(HttpServletRequest request){ //处理图像文件
        DiskFileItemFactory factory = new DiskFileItemFactory();
        ServletFileUpload upload = new ServletFileUpload(factory);
        List<FileItem> items = upload.parseRequest(request);
        Iterator<FileItem> iter = items.iterator();
        while (iter.hasNext()) {
            FileItem item = iter.next();
            String fileName = item.getName();
            String fileEnd = fileName.substring(fileName.lastIndexOf(".") + 1).toLowerCase();
            if(fileEnd != null && fileEnd.matches(regex)){ //依赖文件扩展名进行验证
                //对文件的相关操作
                ...
            }
        }
    }
}

```

示例 2：规范用法之一

```

import org.apache.commons.fileupload.servlet.ServletFileUpload;
import org.apache.commons.fileupload.disk.DiskFileItemFactory;
public class Example{
    private boolean isImage(byte[] buffer){ //判断是否为图像文件
        ...
    }
    public void imageBeauty(HttpServletRequest request){ //处理图像文件
        DiskFileItemFactory factory = new DiskFileItemFactory();
        ServletFileUpload upload = new ServletFileUpload(factory);
        List<FileItem> items = upload.parseRequest(request);
        Iterator<FileItem> iter = items.iterator();
        while (iter.hasNext()) {
            FileItem item = iter.next();
            InputStream is = item.getInputStream(); //以流的形式读取文件内容
            byte[] buffer = new byte[28];
            is.read(buffer,0,28);
            if(isImage(buffer)){ //根据读取的文件内容验证文件是否为图像文件
                //对文件的相关操作
                ...
            }
        }
    }
}

```

6.2.9 用户界面错误

6.2.9.1 点击劫持

漏洞描述：网站未禁止被未信任源加载。

漏洞风险：攻击者可构建一个看似无害的网站，通过将目标网站嵌入其中，诱导用户点击后发送未经授权的命令或窃取敏感信息。

修复或规避建议：设置 X-Frame-Options 的值，禁止网页被未信任源加载。

示例 1：不规范用法

```

public class Example{
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        Throws ServletException, IOException{
        //未设置 X-Frame-Options 的值,攻击者可用 iframe 将该页面嵌入恶意网站中
        ...
    }
}

```

示例 2：规范用法之一

```
public class Example{  
    public void doGet(HttpServletRequest request, HttpServletResponse response)  
        Throws ServletException, IOException{  
        //该页面禁止被任何页面加载  
        response.addHeader("x-frame-options", "DENY");  
        ...  
    }  
}
```

附 录 A
(资料性附录)
Java 语言源代码漏洞测试案例

A.1 范围

本案例为第三方机构的测试人员利用自动化静态分析工具开展的 Java 语言源代码漏洞测试案例。本次测试选择 Java 语言开发的开源 web 应用程序“LibrePlan”(版本号为 1.4.1)共 212902 行源代码进行测试。

A.2 测试依据

GB/T 34944—2017《Java 语言源代码漏洞测试规范》。

A.3 测试目的

本次测试的目的如下：

- a) 指导 GB/T 34944—2017《Java 语言源代码漏洞测试规范》的读者理解该标准；
- b) 发现及定位软件源代码中的漏洞。

A.4 测试过程

本案例的测试过程包括测试策划、测试设计、测试执行和测试总结四个阶段。

A.4.1 测试策划

A.4.1.1 确定测试工具

本项目将采用 Checkmarx CxEnterprise 工具进行测试。

A.4.1.2 确定源代码编译环境

本项目的被测源代码是在 Windows 操作系统下通过 Eclipse 4.4 开发的,测试选择的 Checkmarx CxEnterprise 工具支持该编译环境。

A.4.1.3 分析与评估测试风险并制定应对措施

本项目存在以下风险：

- a) 测试工具无法正常对被测源代码进行扫描。

应对措施:查看测试工具记录的日志,查找问题原因,若无法找到问题原因,则联系测试工具厂商提供技术支持。

- b) 由于技术问题等原因导致实际进度滞后于计划进度。

应对措施:若实际进度滞后于计划进度,则在人工分析时增加人员进行测试结果分析。

A.4.1.4 制定测试进度计划

本项目的测试进度安排见表 A.1。

表 A.1 LibrePlan 源代码漏洞测试进度安排表

测试阶段	人员角色	职责	时间(工作日)	工作成果
测试策划	测试项目负责人	负责项目的组织管理、资源保障和过程评审,明确项目规划,制定测试进度计划	1	《测试计划》
测试设计	测试设计员	开展测试需求分析,确定测试内容、测试方法、测试环境和测试工具,编写测试用例	3	《测试说明》
测试执行	测试员	执行测试,记录测试过程和结果	1	《测试日志》
	测试分析员	人工分析测试结果	2	
测试总结	测试项目负责人	对整个源代码漏洞测试过程进行总结	2	《测试总结报告》

A.4.1.5 形成测试计划

整理以上 A.4.1.1~A.4.1.4 的内容,形成《测试计划》文档。

A.4.1.6 评审测试策划

组织管理层对该项目的测试策划进行评审,评审结果如下:

- a) 测试计划明确了测试需要的工具和环境,测试实施条件要素考虑全面合理;
- b) 测试计划明确的测试人员分工和进度计划具有可实施性;
- c) 测试计划中的测试风险分析全面合理,且具有可行的应对风险的措施。

A.4.2 测试设计

A.4.2.1 分析测试需求

LibrePlan 是 Java 语言开发的 B/S 架构网站,该软件可通过互联网进行访问,容易受到来自互联网的恶意攻击。经分析,Java 语言开发的 B/S 架构的软件源代码容易出现“行为问题”“路径错误”“数据处理”“安全功能”“时间和状态”和“Web 问题”共 6 类 30 个指标的漏洞,这些漏洞是该软件受到恶意攻击的技术根源。本次将重点针对以上 6 类源代码漏洞进行测试。

A.4.2.2 确定测试内容

根据以上测试需求分析,本项目将对以下 Java 语言源代码漏洞进行测试:

- a) 行为问题
不可控的内存分配。
- b) 路径错误
不可信的搜索路径。

- c) 数据处理
 - 1) 相对路径遍历；
 - 2) 绝对路径遍历；
 - 3) 命令注入；
 - 4) SQL 注入；
 - 5) 代码注入；
 - 6) 进程控制；
 - 7) 信息通过错误消息泄露；
 - 8) 信息通过服务器日志文件泄露；
 - 9) 信息通过调试日志文件泄露；
 - 10) 信息通过持久 cookie 泄露；
 - 11) 未检查的输入作为循环条件；
 - 12) XPath 注入。
- d) 安全功能
 - 1) 依赖 referer 字段进行身份鉴别；
 - 2) Cookie 中的敏感信息明文存储；
 - 3) 敏感信息明文传输；
 - 4) 安全关键的行为依赖反向域名解析；
 - 5) 没有要求使用强口令；
 - 6) 没有对口令域进行掩饰；
 - 7) 依赖未经验证和完整性检查的 cookie；
 - 8) 通过用户控制的 SQL 关键字绕过授权；
 - 9) HTTPS 会话中的敏感 cookie 没有设置安全属性。
- e) 时间和状态
 - 1) 会话固定；
 - 2) 会话永不过期。
- f) Web 问题
 - 1) 跨站脚本；
 - 2) 跨站请求伪造；
 - 3) HTTP 响应拆分；
 - 4) 开放重定向；
 - 5) 依赖外部提供的文件的名称或扩展名。

A.4.2.3 确定测试方法

本项目将首先采用自动化静态分析工具对被测源代码进行漏洞扫描,然后人工分析自动化静态分析工具的扫描结果,筛除误报的源代码漏洞。

A.4.2.4 明确测试工具

本项目采用的自动化静态分析工具名称和版本号为“Checkmarx CxEnterprise 6.2.0”。

A.4.2.5 明确测试环境

本项目的测试环境见表 A.2。

表 A.2 LibrePlan 源代码漏洞测试环境

硬件环境		配置(CPU/内存/硬盘/…)	软件环境
设备 编号	名称型号		
H-620	惠普 8560w 移动工作站	Intel Core i7-2720QM (2.2 GHz)/ 16 G/ 750 G	Windows 7 Professional 中文版(64 位) SQL Server 2008 Express Edition(64 位) IIS 7 IE8 Checkmarx CxEnterprise 6.2.0(测试工具)

A.4.2.6 设计测试用例

本项目的测试用例见表 A.3。

表 A.3 LibrePlan 源代码漏洞测试用例

测试用例编号		YMLD-YL-2016-1001-0001	
测试用例名称		LibrePlan 源代码漏洞测试用例	
编号	测试内容、测试过程和测试方法		期望结果
1	使用 IE8 浏览器打开 Checkmarx CxEnterprise 6.2.0		显示测试工具的登录 页面
2	在“登录”页面中输入用户名(administrator@cx)和密码(Sb3396981 * * *),点击“登录”按钮		成功登录工具页面
3	在工具页面中选择“行为问题”类的“不可控的内存分配”指标进行 测试		明确该源代码是否存在 “不可控的内存分 配”指标的漏洞
…	其余 5 类 29 个指标的测试内容、测试过程和测试方法同上,此处略		…
测试用例设计人员		测试用例设计日期	××××-××-××

A.4.2.7 形成测试说明

整理以上 A.4.2.1~A.4.2.6 的内容,形成《测试说明》文档。

A.4.2.8 评审测试设计

组织管理层对该项目的测试设计进行评审,评审结果如下:

- a) 测试需求分析合理;
- b) 测试内容和方法符合测试需求;

- c) 测试用例的操作步骤和输入数据详细、正确、可实施；
- d) 测试用例的期望结果描述准确；
- e) 测试人员、环境和工具齐备并符合要求。

A.4.3 测试执行

A.4.3.1 执行测试并记录测试过程和结果

根据测试用例执行测试,测试记录见表 A.4。

表 A.4 LibrePlan 源代码漏洞测试记录

测试记录编号	YMLD-JL-2016-1001-0001		
测试用例名称	LibrePlan 源代码漏洞测试记录		
编号	测试内容、测试过程和测试方法		实际结果
1	使用 IE8 浏览器打开 Checkmarx CxEnterprise 6.2.0		打开测试工具的登录页面
2	在“登录”页面中输入用户名(administrator@cx)和密码(Sb3396981 * * *),点击“登录”按钮		登录至测试工具的主页面
3	在工具页面中选择“行为问题”类的“不可控的内存分配”指标进行测试		未发现“不可控的内存分配”源代码漏洞
...	其余 5 类 29 个指标的测试内容、测试过程和测试方法同上,此处略		发现其余 5 类 29 个指标共 240 个源代码漏洞
测试执行人员		测试执行日期	××××-××-××

A.4.3.2 分析测试结果

通过人工分析发现,自动化静态分析工具由于不识别部分对用户输入进行过滤的函数导致出现 9 个误报,误报的源代码漏洞已在自动化静态分析工具中进行了标记和说明。

A.4.3.3 形成测试日志

整理以上 A.4.3.1 和 A.4.3.2 的内容,形成《测试日志》文档。

A.4.3.4 评审测试执行

组织管理层对该项目的测试执行进行评审,评审结果如下:

- a) 测试用例执行已达到 100%；
- b) 操作结果真实有效；
- c) 操作结果描述清晰、准确；
- d) 本次测试未出现与预期结果不一致的操作结果；
- e) 经复核,人工分析的结果正确。

A.4.4 测试总结

A.4.4.1 编制测试总结报告

在本次测试项目周期内,对 LibrePlan 项目 212902 行源代码,依据 GB/T 34944—2017《Java 语言

源代码漏洞测试规范》，采用自动化静态分析工具扫描和人工分析相结合的方法，进行源代码漏洞测试。本次源代码漏洞测试的测试内容和结论如下：

本次源代码漏洞采用“Checkmarx CxEnterprise 6.2.0”测试工具，对不可控的内存分配、不可信的搜索路径等 30 种 Java 语言源代码漏洞进行了扫描，共扫描出 240 个源代码漏洞。经人工对漏洞进行分析，发现部分对用户输入进行过滤的函数来源于第三方可执行代码库，自动化静态分析工具由于不识别这些函数导致出现 9 个误报。筛除误报后仍存在 231 个漏洞。

本次测试的详细测试结果见表 A.5。

表 A.5 LibrePlan 源代码漏洞测试结果

序号	测试内容	测试结果	备注
1	不可控的内存分配	未发现漏洞	
...	其余 5 类 29 个指标内容略	发现其余 5 类 29 个指标共 231 个源代码漏洞	源代码漏洞的详情可到自动化静态分析工具中查看

A.4.4.2 评审测试总结

组织管理层对该项目的测试总结进行评审，评审结果如下：

- a) 测试需求和测试目标已完成；
- b) 测试结论能够追溯到测试结果；
- c) 已具备测试准出条件；
- d) 测试过程中风险规避合理。

注：本案例相关内容的格式仅供参考。

附 录 B
(资料性附录)

Java 语言源代码漏洞类别与名称

为方便读者查阅本标准的源代码漏洞,以下按本标准漏洞出现的顺序列出了本标准所有漏洞的类别和名称:

- a) 行为问题
 - 不可控的内存分配。
- b) 路径错误
 - 不可信的搜索路径。
- c) 数据处理
 - 1) 相对路径遍历;
 - 2) 绝对路径遍历;
 - 3) 命令注入;
 - 4) SQL 注入;
 - 5) 代码注入;
 - 6) 进程控制;
 - 7) 信息通过错误消息泄露;
 - 8) 信息通过服务器日志文件泄露;
 - 9) 信息通过调试日志文件泄露;
 - 10) 信息通过持久 cookie 泄露;
 - 11) 未检查的输入作为循环条件;
 - 12) XPath 注入。
- d) 处理程序错误
 - 未限制危险类型文件的上传。
- e) 不充分的封装
 - 1) 可序列化的类包含敏感数据;
 - 2) 违反信任边界。
- f) 安全功能
 - 1) 明文存储口令;
 - 2) 存储可恢复的口令;
 - 3) 口令硬编码;
 - 4) 依赖 referer 字段进行身份鉴别;
 - 5) Cookie 中的敏感信息明文存储;
 - 6) 敏感信息明文传输;
 - 7) 使用已破解或危险的加密算法;
 - 8) 可逆的散列算法;
 - 9) 密码分组链接模式未使用随机初始化矢量;
 - 10) 不充分的随机数;

- 11) 安全关键的行为依赖反向域名解析;
 - 12) 关键参数篡改;
 - 13) 没有要求使用强口令;
 - 14) 没有对口令域进行掩饰;
 - 15) 依赖未经验证和完整性检查的 cookie;
 - 16) 通过用户控制的 SQL 关键字绕过授权;
 - 17) HTTPS 会话中的敏感 cookie 没有设置安全属性;
 - 18) 未使用盐值计算散列值;
 - 19) RSA 算法未使用最优非对称加密填充。
- g) 时间和状态
- 1) 会话固定;
 - 2) 会话永不过期。
- h) Web 问题
- 1) 跨站脚本;
 - 2) 跨站请求伪造;
 - 3) HTTP 响应拆分;
 - 4) 开放重定向;
 - 5) 依赖外部提供的文件的名称或扩展名。
- i) 用户界面错误
- 点击劫持。

参 考 文 献

- [1] GB/T 25069—2010 信息安全技术 术语
 - [2] GB/T 28458—2012 信息安全技术 安全漏洞标识与描述规范
 - [3] CWE-Common Weakness Enumeration. <http://cwe.mitre.org>
 - [4] Code Conventions for the Java Programming Language. <http://www.oracle.com/technet-work/java/codeconvtoc-136057.html>
-